The Arden Syntax for Medical Logic Modules

Article *in* International Journal of Clinical Monitoring and Computing · December 1993

DOI: 10.1007/BF01133012 · Source: PubMed Central

6 authors, including:

Peter J Haug
Intermountain Medical Center
214 PUBLICATIONS **4,180** CITATIONS
SEE PROFILE

Ove Wigertz
Linköping University
114 PUBLICATIONS **1,489** CITATIONS
SEE PROFILE

Johan van der Lei
Erasmus University Rotterdam
232 PUBLICATIONS **6,413** CITATIONS
SEE PROFILE

The Arden Syntax for Medical Logic Modules

George Hripcsak[1], Paul D. Clayton[1], T. Allan Pryor[2],
Peter Haug[2], Ove B. Wigertz[3], Johan Van der lei[4]

1. Center for Medical Informatics, Columbia-Presbyterian Medical Center, New York, NY 10032. 2. Dept. of Medical Informatics, LDS Hospital/University of Utah, Salt Lake City, UT 84143. 3. Dept. of Medical Informatics, Linkoping University, S581 83 Linkoping, Sweden. 4. Dept. of Medical Informatics, Erasmus University, 3000 DR Rotterdam, The Netherlands.

The Arden Syntax for sharing medical knowledge bases is described. Its current focus is on knowledge that is represented as a set of independent modules that can provide therapeutic suggestions, alerts, diagnosis scores, etc. The syntax is based largely upon HELP and the Regenstrief Medical Record System. Each module, called a Medical Logic Module or MLM, is made of slots grouped into maintenance, library, and knowledge categories. The syntax has provisions for querying a clinical database and representing time. Several clinical information systems were analyzed and appear to be compatible with the syntax. The syntax has been tested for syntactic ambiguities using the tools lex and yacc. Four institutions are currently in the process of adopting the Arden Syntax for their decision-support systems.

INTRODUCTION

Medical decision-support systems have been used successfully for a number of years. Several institutions have already assembled large medical knowledge bases that cover parts of medicine [1-4]. Since no one institution will ever define a complete medical knowledge base, it will be necessary to share complementary knowledge bases among institutions. Unfortunately, there are many obstacles to sharing, including coordinating local vocabularies, setting local prior probabilities or weights, integrating existing systems that already use some form of decision-support, translating the logic for use on disparate computers systems, keeping the knowledge bases current, linking knowledge to the literature, liability, and royalties [5-8]. Work has begun along several avenues to overcome these obstacles [5]. This paper addresses a single avenue, the definition of a common syntax, called the Arden Syntax, for expressing medical knowledge.

The Arden Syntax is named after the Arden Homestead in Harriman, NY, where an early retreat with representatives from ten universities was held to address the sharing of knowledge [5]. There is insufficient space to describe the syntax exhaustively. Instead this paper focuses on the motivations for designing a new syntax, features of the syntax that directly affect sharing, and the current status of the verification of the syntax.

SCOPE

There are many different types of medical knowledge and many different types of decision-support systems. A project that attempts to accommodate everything is doomed. Therefore, the initial scope of the project has been limited to knowledge that can be represented in an easily shared format.

Knowledge that can be represented as modular units that are relatively independent from each other is most amenable to sharing. The modular units are referred to as Medical Logic Modules (MLM's). Many types of medical decision-support are amenable to representation by independent modules. For example, alerts, therapeutic suggestions, management critiques, and diagnostic scoring algorithms have been encoded in this fashion [1,2,9].

If the MLM's are independent, then they can be added to an institution's system one by one. The institution may benefit from the use of even a single MLM if it contains a useful alert or suggestion. References to the patient database can be resolved and checked, and local prior probabilities can be adjusted one MLM at a time. This is easier than trying to incorporate a comprehensive decision-support system at once.

Note that the use of knowledge bases for multiple purposes is currently outside the scope of the syntax. For example, a management suggestion would be shared for use as a management suggestion, rather than for its potential use in computer aided instruction.

SYNTAX ALTERNATIVES

Given these aims and scope, it must be decided whether to adopt an existing syntax, or to define a new syntax for sharing knowledge. There are several alternatives.

General-purpose programming languages are certainly expressive enough to define medical logic. The main problem is that they lack medically relevant constructs and data structures like database queries and time. If each institution is free to implement these items as it pleases (e.g., its own database implementation of time), then sharing between institutions will require significant translation.

General purpose query languages like SQL can retrieve data but do not contain the constructs needed to compare data in complex ways. For example, SQL lacks the block structure that would be provided by an IF-THEN statement, and it lacks constructs for time.

Other alternatives include the syntaxes of existing decision-support systems. The syntax of Health Evaluation through Logical Processing (HELP) [10] and the syntax of the Regenstrief Medical Record System (RMRS) [2] are of particular interest because both support modular independent knowledge, both have demonstrated success, and both are well established in their sponsoring institutions. One problem is that each of these systems supports queries only to its own proprietary patient database. The queries have not been designed specifically for the purpose of sharing knowledge. Furthermore, some systems (like HELP) contain constructs for purposes other than for making medical decisions. For example, there are constructs for report generation and application development. These facilities are beyond the current scope of MLM's. Last, to foster sharing, one wants an open, public standard. There is a danger of using a language that is proprietary and that may be linked to a specific vendor.

ARDEN SYNTAX OVERVIEW

The Arden Syntax for MLM's has been designed specifically to share medical knowledge. There are constructs for database queries that are intended to facilitate mapping them to an institution's patient database; there are constructs and data structures for time; and there are provisions for tracking changes in MLM's and assigning responsibility for the use of an MLM in

This work was supported in part by a grant from the National Library of Medicine LM04419 (IAIMS). The Arden Homestead Retreat was sponsored by the CAMDAT Foundation, IBM, and the law firm of De Forest and Duer.

0195-4210/90/0000/0200$01.00 © 1990 SCAMC, Inc.

an institution. The syntax is derived largely from HELP, which resembles Pascal. In addition, several features in RMRS have been incorporated; a number of its aggregation operators are included, and the manner in which the result of a database query can be manipulated has been adopted. Although other systems like Quick Medical Reference (QMR) [3] and DXplain [4] have been assessed for the potential of expressing their knowledge bases in the Arden Syntax, they have not yet been incorporated into the syntax.

Refer to Figure 1 for a sample MLM. It is derived from a previously published sample MLM [5]. The minor differences that have occurred in the syntax reflect insights gained from testing and implementing the syntax over the past year (an example is given in a subsequent section).

Basic Structure

An MLM is a text file that is arranged into discrete slots, much like HELP and RMRS. A slot is composed of a slot name (e.g., "title:") and a slot body. Depending on the slot, the slot body may contain free text, a coded term, or structured data. In order to make the MLM more readable, the slots are grouped into three categories: maintenance, library, and knowledge.

Maintenance Category

The maintenance category contains those slots that specify maintenance information that is unrelated to the medical knowledge in the module. The title, filename, author, and institution slots serve to name the MLM and identify its source.

The specialist slot names the person in the institution who is responsible for validating and installing the MLM in the institution. This person sets a validation level in the validation slot: production, research, testing, or expired. The specialist slot should always be blank when transferring MLM's from one institution to another; it is the borrowing institution's responsibility to fill this slot and accept responsibility for the use of the MLM.

Simple version control is accomplished using the version and date slots. Updates to the MLM are tracked using a version number, the date of creation of the MLM, and the date of last revision. As MLM's are revised, old versions are archived.

Library Category

The library category contains those slots pertinent to maintenance that are related to the module's knowledge. The purpose slot serves as a comment. The keywords slot contains descriptive words used for searching through modules and for searching through the literature. Unified Medical Language System (UMLS) terms [11] are preferred but not mandatory.

References are listed in the citations slot. Comments can appear anywhere in the MLM; those that contain only a number (e.g., "/*4*/") refer to the citation of that number. Links to other sources, such as an electronic textbook, teaching cases, or educational modules may be placed in the links slot.

Knowledge Category

The knowledge category contains the actual medical knowledge of the MLM. The type slot identifies the way in which the MLM is used. The data slot defines terms used in the rest of the MLM. A short name (i.e., a local variable) is substituted for a longer database query.

The evoke slot contains the conditions under which the MLM becomes active. One can view it as setting the context for the MLM. For example, the storage of particular data elements in the patient database may evoke the MLM. The logic slot contains the actual medical rule or medical condition to test for. This slot can contain complex calculations. The action slot specifies what is to be done if the premise is satisfied. Possibilities include sending an alert to a destination, evoking other MLM's, and returning values.

```
maintenance:
   title:  Agranulocytosis and
       Trimethoprim/Sulfamethoxazole;
   filename:   anctms;
   version:    2.00;
   institution:    Zoo University;
   author: Dr. Bonzo;
   specialist: ;
   date:   7/20/1989, 7/23/1990;
   validation: testing;

library:
   purpose:    Display the Arden Syntax;
   keywords:
       granulocytopenia; agranulocytosis;
       trimethoprim; sulfamethoxazole;
   citations:
       1.  Anti-infective drug use in
       relation to the risk of
       agranulocytosis and aplastic
       anemia....  Arch Int Med
       1989;149(5):1036-40.
   links:
       MeSH agranulocytosis/ci and
       sulfamethoxazole/ae;

knowledge:
   type:   data-driven;
   data:
       /* neutrophile count in #/mm3 */
       anc := read last 2 from ({query
           for ANC} where it occurred
           within the past 1 week);
       pt_taking_tms := read exist
           {query for TMS order};
   evoke:  on storage of {ANC};
   logic:
       if pt_taking_tms        /*1*/
       and last anc < 1000
       and decrease of anc > 0 then
           conclude true
       else
           conclude false;
   action: store "Caution:  The patient's
       relative granulocytopenia may be
       exacerbated by
       trimethoprim/sulfamethoxazole.";
```

FIGURE 1. A SAMPLE MLM

The maintenance and library sections help maintain the MLM. The knowledge section contains the actual logic. The evoke slot specifies that this MLM is evoked whenever an absolute neutrophile count (ANC) is stored. In the data slot, the local variable "anc" is assigned the patient's last 2 ANC's within the past week. "pt_taking_tms" is assigned true or false depending on whether the patient is taking trimethoprim and sulfamethoxazole (TMS). The logic slot says that if the patient is taking TMS and if the last ANC is less than 1000 and if the ANC is decreasing, then execute the action slot, which sends the appropriate message.

The syntax of the data, evoke, logic, and action slots is similar to Pascal. For example,

```
3 + 4*2
```

is a numeric expression (it equals 11). Figure 1 contains other examples in the logic slot.

The Arden Syntax follows RMRS's use of control statements. RMRS uses IF-THEN statements and an EXIT

instruction, but it lacks constructs for looping and branching [2]. The result is a more declarative syntax that is easier to implement and that can be easier to read by those unfamiliar with computer languages. Although HELP does include looping constructs, they are used primarily for result reporting and application development rather than for medical decision logic.

The Arden Syntax includes database queries, constructs for time, and lists, all of which are discussed in the next section.

DATABASE QUERIES

Why Queries Are Essential

"The use of digoxin in the setting of hypokalemia may cause cardiac arrhythmias" is a simple rule. But applying this rule to a particular patient is difficult. Given the rule and a patient's chart, a physician will use common sense when interpreting laboratory values. If the last serum potassium is hemolyzed, the physician will look for the previous one. If the last serum potassium was performed four years ago, then the physician will not use that value. If the value for a recent potassium is 56.6, then the physician will suspect that there has been a laboratory error.

Many decision-support systems are able to collect data from a computerized patient database without human intervention (e.g., HELP, RMRS, and ALERTS [9]). This ability obviates the need for health care providers to re-enter data that is already stored electronically; it permits MLM's that check for contraindications to run in the background without disturbing a health care provider until an alert is actually generated; and it supports the triggering of MLM's without a specific request for assistance [12]. Unfortunately, since such systems lack a human being to filter queries, their MLM's must foresee every contingency for every query to the database.

This section focuses on a critical part of the Arden Syntax: database queries. For each feature in the syntax of the queries, two factors are important: whether the feature is useful to medical rules and whether the feature can be implemented on patient databases in existing clinical information systems. To assess usefulness, several automated decision-support systems were reviewed. Those that use modular independent rules and that are linked to an active patient database were selected for the most detailed study. These included HELP [1,10], RMRS [2,13], and ALERTS [9]. To assess the feasibility of the query syntax, several well-known clinical information systems were reviewed: HELP, RMRS, Computer Stored Ambulatory Record (COSTAR) [14], The Medical Record (TMR) [15], Summary Time Oriented Record (STOR) [16], Patient Care System (PCS) [17], and the Beth Israel Hospital and Brigham and Women's Hospital clinical computing systems [18,19].

Disparate Vocabularies and Database Structures

One of the more difficult problems of sharing an MLM between two institutions is the difference in the vocabularies and database structures used in the institutions [5-7]. Most institutions' vocabularies are a unique mixture of local nomenclature, terms used by vendors in purchased systems, and standard vocabularies like ICD-9. The review of the clinical information systems revealed such diversity in vocabulary and database structure [1,13-19] that the definition of a single syntax for querying patient databases was not attempted. The UMLS [11] has made progress in mapping between standard vocabularies, but automatically translating queries to accommodate disparate patient databases and mapping terms to local clinically descriptive vocabularies is probably not feasible at the current time.

Instead, queries are split into two components: an institution-specific component phrased in the authoring institution's own query language, and a more general component phrased in the Arden Syntax. When MLM's are shared, the receiving institution must manually translate the institution-specific component into its own query language.

Following the example of HELP [10] and RMRS [2], access to institution-specific data has been extracted from the logic and placed in a separate data slot. This serves to isolate the part of an MLM that will generally need to be changed when an MLM is shared from the rest, which will hopefully need little alteration (the degree to which logic can be written in multiple centers and shared among several institutions has been the subject of several studies (e.g., [20,21]) and results so far are promising). Within the data slot, institution-specific queries are mapped to local names used elsewhere in the MLM. As long as the local names have been assigned the same data elements, the logic of the MLM may undergo little change despite major changes to the queries.

Availability of Data

If an MLM requires a data element that is not available in an institution's database, then the MLM cannot be used without modification. Sometimes a missing data element can be derived from other data that are stored in the database. For example, an MLM that requires a patient's systemic vascular resistance can be used by an institution that records mean arterial pressure and cardiac output, from which the systemic vascular resistance can be calculated.

The data slot has provisions to perform calculations on the results of queries. Then the result of the calculation can be assigned to the local variable name used in the logic of the MLM. In this way, the logic need not know whether its data came from a direct query or a calculation on a set of queries.

Aggregation Operators

An aggregation operator performs a summary function on a set of data based on some property of the data; examples include MAXIMUM, LAST, and SUM. Aggregation has been found to be useful in medical rules in HELP [10], RMRS [2], and ALERTS [9].

A database management system (DBMS) can support aggregation in several ways. First, it can provide aggregation directly through its own query language, as do relational databases that use SQL [22]. Second, it can provide aggregation indirectly by supplying primitive operations that can be assembled to perform aggregation in an efficient manner. A DBMS that can retrieve the last occurrence of a patient attribute and then iteratively retrieve preceding instances is sufficient to perform all the aggregation operations in the Arden Syntax except FIRST. Third, the aggregation operator can be applied by support routines after the query is completed. This third alternative is not feasible for operators like LAST for efficiency reasons; one would not want to retrieve all instances of some patient attribute just to use the last one. All of the reviewed clinical information systems appeared to support at least some form of aggregation [1,13-19]. It is likely that missing operators could be added to these systems by writing the appropriate support routines.

There are relatively few aggregation operators, so agreeing on a common syntax for them is not as difficult as it is for general medical terms. The aggregation operators have been separated out from the rest of the query, and a standard syntax has been defined for them.

Time

. HELP [10], RMRS [2], and ALERTS [9] use a temporal model that supports discrete points in time (e.g., "now" or "13:12:13 1990-02-28"), and durations, which are periods of time (e.g., "2 months" or "3 days"). These elements can be put together to form more complex expressions:

3 months ago
4 weeks before now
2 days after the time of surgery

Operators use these elements to create search windows:

... where it occurred within the past 3 months

All of the patient databases that were reviewed stored at least discrete time points along with the data elements, and should be capable of supporting the operations in the model [1,13-19].

More sophisticated models of time were considered. (For example, STOR records discrete time values along with an associated value that indicates the precision with which the time value is known [16].) But because of its common usage, simplicity of design, ease of implementation, and success in the above mentioned decision-support systems, the discrete temporal model is used in the Arden Syntax.

Like aggregation operators, the time constraints have a limited vocabulary, and mapping them to an institution's query language need not be difficult. They, too, have been separated from the rest of the query, and a standard syntax has been defined for them.

Query Syntax

The general form of a query is:

<var> := READ <aggregation> ({ <body> }
WHERE <t_constraint>)

where <var> is a local variable; <aggregation> is one or more aggregation operators; <body> is the institution-specific database query devoid of aggregation and time constraints; and <t_constraint> is the temporal search window.

For example:

k := READ THE MAXIMUM ({select potas from electro} WHERE IT OCCURRED WITHIN THE PAST 3 DAYS);

The local variable "k" is assigned the maximum serum potassium within the past three days. "MAXIMUM" is an aggregation operator that picks the highest potassium value. The phrase "select potas from electro" is the institution-specific body of the query, devoid of aggregation and time constraints, written in the query language of the institution's DBMS (a relational database in this example). "WHERE IT OCCURRED WITHIN THE PAST 3 DAYS" is the temporal search window for the query.

When an institution wants to run an MLM, it must compile the high-level form into a form understandable to its computer system. A part of the compilation process is to insert the aggregation and time constraints into the body of the query using the institution's query language. For example, a compiler would automatically turn the last query into this:

k := READ {select max(potas) from electro where time > (now - 3 days)};

assuming that "time" is a valid column and "days" is a valid term in the query language. This new body could then be executed by the institution's DBMS. This version of the query need not be altered or even seen by a human being.

When another institution borrows the MLM, someone will have to alter it manually to fit its patient database. If the institution uses a completely foreign database structure, the altered query may look like this:

k := READ THE MAXIMUM ({123 from 456 to 460} WHERE IT OCCURRED WITHIN THE PAST 3 DAYS);

Although the institution-specific body changes, the aggregation operators and the time constraint do not need to be changed. The institution's compiler would then insert the aggregation and time constraints into the body at the time of compilation.

Whether it is possible to separate out the aggregation and time constraints and then to automatically insert them depends on the DBMS. Two systems have been studied in detail so far: a relational database which uses SQL and a hierarchical database which uses IBM's DL/I query language along with a set of support routines. A compiler that automatically inserts these elements has been completed for both systems.

It remains a question whether other systems will be able to do the same. At worst, the entire query can be written in the institution's query language when aggregation and time constraints cannot be separated logically from the rest of the query. This creates more work on transferring the MLM, but it accommodates a wider range of DBMS's.

Lists

The results of these queries are used in the logic slot. Unless specific aggregation operators are used, one cannot know how many items will be returned by a query. For example, even if a query requests a patient's age, multiple values may be returned because of conflicting evidence stored in the database.

Both RMRS and HELP allow several items to be returned as the result of a query: RMRS does so by default, and HELP uses its "relation" construct. Similarly, the Arden Syntax allows local variables to hold lists of items. For example:

potas := {select potas from electro}

would place all the serum potassiums a patient ever had in the local variable "potas."

The resultant list can be manipulated in the logic slot using aggregation operators much like the ones in database queries. Continuing with the above example, the following expression in the logic slot:

MAXIMUM potas / AVERAGE potas

would result in the ratio between the largest potassium value in the list and the average potassium value. The selection operator, WHERE, can also be applied to lists. The following expression:

potas WHERE IT > 5

would select only those potassium values that are greater than 5.

TESTING AND IMPLEMENTATION

Syntactic Verification

The Arden Syntax has been tested for consistency and ambiguity using the tools lex [23] and yacc [24]. Used together, these tools will generate a compiler given a high-level syntactic specification. In the process they produce a listing of conflicts (e.g., shift-reduce, reduce-reduce [25]) that represent ambiguities in the syntax. The Arden Syntax has gone through a process of testing and modification to remove unforeseen syntactic ambiguities.

For example, in the data slot in Figure 1, the first query contains the time constraint "where it occurred within the past 1 week". This replaced the previous version, "between now and 1 week before now" [5], because the keyword AND was used for logical AND and for the BETWEEN-AND operator, and this caused a conflict.

Implementation

The only true test of the syntax is to use it. Both its ability to represent medical knowledge within an institution and its amenability to sharing must be judged. At this point, the syntax is being implemented at four centers.

At Columbia-Presbyterian Medical Center, New York, New York, a medical decision-support system that uses a subset of the Arden Syntax has been completed [26]. It applies MLM's to the institution's existing patient database. At this stage, the generated alerts are not yet being sent to health care providers.

At LDS Hospital, Salt Lake City, Utah, the HELP system has been functioning for many years, and there are already

several thousand MLM's. The current focus is to bring the Arden Syntax and the decision-support component of the HELP language into agreement.

At Linkoping University, Linkoping, Sweden, a medical decision-support system is being developed [27]. It includes the clinical database, the medical data dictionary, and the knowledge base component. The Arden Syntax will be the syntax for the knowledge base.

The group at Erasmus University, Rotterdam, the Netherlands, performed much of the syntactic verification. The present focus is on implementing the Arden Syntax using MUMPS and utilities similar to lex and yacc.

An ASTM (American Society of Testing Materials) subcommittee on sharing computerized medical knowledge bases is being established under the chairmanship of Dr. Allan Pryor. It will be the forum for involvement by other institutions, it will be a vehicle for dissemination of the syntax, and it will provide support in matters such as copyrighting.

CONCLUSION

The routine sharing of medical knowledge bases is a large and very important project. By limiting the scope of the project to modular independent knowledge and by basing it on existing systems, such sharing of knowledge may become feasible. The Arden Syntax is designed to facilitate sharing medical knowledge. The knowledge is contained in independent units called MLM's. They include information necessary to track and maintain the knowledge base. The syntax for database queries is intended to ease the transfer of MLM's between institutions. Features that have been found to be useful in existing decision-support systems have been incorporated into the queries, and it appears that clinical information systems will be able to accommodate them. Perhaps most important, the Arden Syntax is an public standard, and a new ASTM subcommittee will be responsible for the syntax.

Working systems that use the standard have been and are being constructed. Copies of the complete syntax can be obtained from George Hripcsak.

REFERENCES

[1] Pryor TA, Gardner RM, Clayton PD, Warner HR. The HELP system, In Blum BI ed., Information Systems For Patient Care, New York: Springer Verlag, 1984; 109-128.

[2] McDonald CJ. Action-Oriented Decisions in Ambulatory Medicine. Chicago: Year Book Medical Publishers, 1981.

[3] Miller RA, McNeil MA, Challinor SM, Masarie FE, Myers JD. The INTERNIST-1/Quick Medical Reference report. West J Med 1986;145:816-822.

[4] Barnett GO, Cimino JJ, Hupp JA, Hoffer EP. DXplain: an evolving diagnostic decision-support system. JAMA 1987;258(1):67-74.

[5] Clayton PD, Pryor TA, Wigertz OB, Hripcsak GM. Issues and structures for sharing knowledge among decision-making systems: The 1989 Arden Homestead Retreat. Kingsland LC ed., Proc 13th SCAMC, IEEE Comp Soc Press, 1989; 116-121.

[6] Clayton PD, Pryor TA, Wigertz OB, Johnson SB, Hripcsak GM. Sharing medical knowledge for automated decision-making. Greenes RA ed., Proc 12th SCAMC, IEEE Comp Soc Press, 1988; 591-594.

[7] Wigertz OB, Clayton PD, Hripcsak G, Linnarsson R. Knowledge representation and data model to support medical knowledge base transportability. In Talmon J, Spiegelhalter J, System Engineering in Medicine, Heidelberg: Springer Verlag, 1989.

[8] Yasnoff WA. Technology transfers of medical decision assistance systems. Hammond WE ed., AAMSI Congress 89, AAMSI Publishers, 1989; 103-107.

[9] Shabot MM, LoBue M, Leyerle BJ, Dubin SB. Inferencing strategies for automated ALERTS on critically abnormal laboratory and blood gas data. Kingsland LC ed., Proc 13th SCAMC, IEEE Comp Soc Press, 1989; 54-57.

[10] HELP Frame Manual, Version 1.6. Salt Lake City: LDS Hospital, 1989.

[11] Lindberg DAB, Humphreys BL. Toward a unified medical language. Medical Informatics Europe 87, Proceedings of the Seventh International Congress, Rome, 1987;1:23-31.

[12] Clayton PD, Evans RS, Pryor TA, Gardner RM, Haug PJ, Warner HR. Data driven interpretation of laboratory results in the context of a medical decision support system. In Keller H, Trendelenburg C eds., Data Presentation/Interpretation. Berlin: Walter de Gruyter, 1989; 367-380.

[13] McDonald CJ, Blevins L, Tierney WM, Martin DK. The Regenstrief Medical Records. MD Computing 1988;5(5):34-47.

[14] Barnett GO, Zielstorff RD, Piggins J, et al. A comprehensive medical information system for ambulatory care. Blum BI ed., Proc 6th SCAMC, IEEE Comp Soc Press, 1982:8-18.

[15] Stead WW, Hammond WE. Computer-based medical records: the centerpeice of TMR. MD Computing 1988;5(5):48-62.

[16] Whiting-O'Keefe QE, Whiting A, Henke J. The STOR clinical information system. MD Computing 1988;5(5):8-21.

[17] Patient Care System ORDERS: Database Reference Manual. Atlanta: International Business Machines Corporation, 1988.

[18] Safran C, Slack WV, Bleich HL. Role of computing in patient care in two hospitals. MD Computing 1989;6(3):141-148.

[19] Safran C, Porter D, Rury CD, et al. ClinQuery: searching a large clinical database. MD Computing 1990;7(3):144-153.

[20] Giuse NB, Giuse DA, Miller RA. Computer assisted multi-center creation of medical knowledge bases. Greenes RA ed., Proc 12th SCAMC, IEEE Comp Soc Press, 1988; 583-590.

[21] Lincoln MJ, Turner C, Hesse B, Miller R. A comparison of clustered knowledge structures in Iliad and in Quick Medical Reference. Greenes RA ed., Proc 12th SCAMC, IEEE Comp Soc Press, 1988; 131-135.

[22] Date CJ. An Introduction to Database Systems, volume 1. Reading: Addison-Wesley Publishing Company, 1986.

[23] Lesk ME, Schmidt E. Lex - a lexical analyser generator. Computer Science Technical Report No. 39. Murray Hill: Bell Labs., 1975.

[24] Johnson SC. Yacc: yet another compiler compiler. Computer Science Technical Report No. 32. Murray Hill: Bell Labs., 1975.

[25] Tremblay, J-P, Sorenson PG. The Theory and Practice of Compiler Writing. New York: McGraw-Hill Book Co., 1985.

[26] Hripcsak G, Clayton PD, Cimino JJ, Johnson SB, Friedman C. Medical Decision Support at Columbia-Presbyterian Medical Center. IMIA Working Conference on Software Engineering in Medical Informatics, Amsterdam, 8-10 October 1990.

[27] Magyar G, Arkad K, Ericsson K-E, Gill H, Linnarsson R, Wigertz O. Realizing medical knowledge in MLM form as working modules in a patient information system. IMIA Working Conference on Software Engineering in Medical Informatics, Amsterdam, 8-10 October 1990.